

DataLink Engine — System & Architecture Manual

Enterprise Real Estate Data Ingestion, Normalization, Deduplication & Export Platform

Python 3.12

FastAPI

React 18

Vite 5

Supabase PostgreSQL

VS Code Tooling

Production v2.0

Document Version: 2.0.0

Status: Production Ready / Verified

Target Audience: Technical Reviewers, Solution Architects, Data Engineers, QA Teams

Primary Development Environment: Visual Studio Code (VS Code)

1. Executive Summary & Objective

DataLink Engine is a high-throughput, fault-tolerant real estate data normalization, enrichment, and ingestion platform. It transforms disparate, messy, multi-builder Excel (`.xlsx` , `.xls`) and CSV spreadsheets into a canonical, outreach-ready relational dataset stored in PostgreSQL (Supabase).

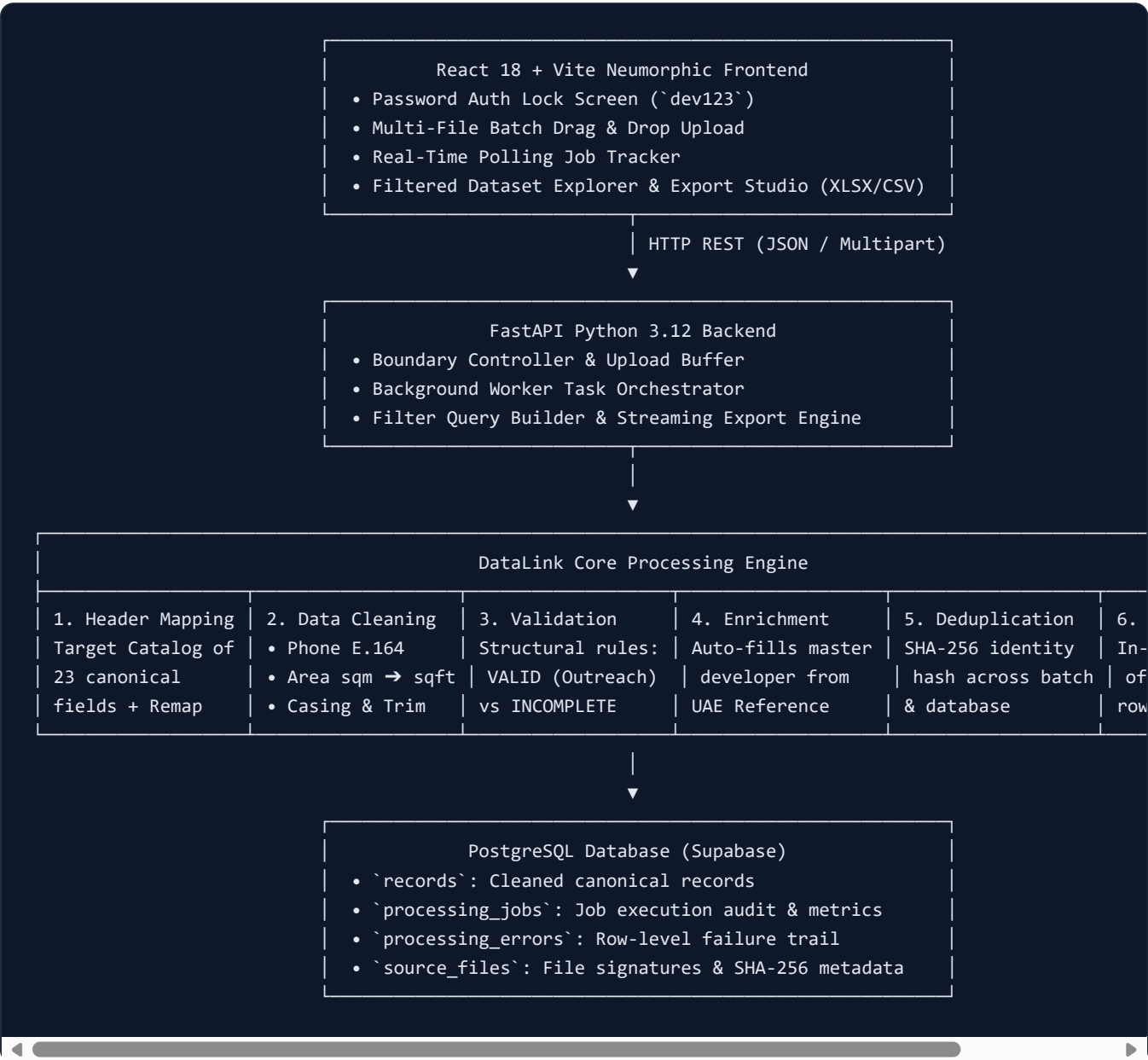
The system operates on a **strictly linear, unidirectional pipeline**: $\text{Raw File} \rightarrow \text{Inspect/Map} \rightarrow \text{Clean} \rightarrow \text{Validate} \rightarrow \text{Enrich (UAE Reference)} \rightarrow \text{Deduplicate} \rightarrow \text{PostgreSQL Storage} \rightarrow \text{Explorer / Export}$

2. Technology Stack & Tooling

The platform was developed and tested using **Visual Studio Code (VS Code)** with a dedicated Python virtual environment (`.venv`) and Node.js toolchain.

LAYER	TECHNOLOGY	VERSION	PURPOSE & RATIONALE
IDE / Tooling	Visual Studio Code (VS Code)	1.85+	Primary development environment for Python, React, SQL, and Git version control.
Frontend UI	React + Vite	React 18, Vite 5.0	High-performance component-driven interface with sub-millisecond HMR and instant state updates.
Design System	Vanilla CSS + Neumorphism	CSS Variables	Tactile White & Dark Slate Neumorphic UI with soft 3D elevation, debossed insets, and high-contrast typography.
Icons	Lucide React	0.344+	Crisp, accessible SVG icon library.
Backend Framework	Python + FastAPI	Python 3.12, FastAPI 0.110+	Asynchronous REST backend with automatic OpenAPI documentation and strict Pydantic v2 data validation.
Web Server	Uvicorn	0.28+	Lightning-fast ASGI web server.
Data Processing	Pandas & OpenPyXL	2.2+, 3.1+	Memory-bounded tabular data streaming, robust header extraction, and multi-sheet parsing.
Database ORM	SQLAlchemy	2.0+	Declarative models, connection pooling, and atomic batch transactions.
Database	PostgreSQL (Supabase)	PG 15+	Transactional ACID storage with JSONB support, composite indexing, and connection pooling.
Cloud Hosting	Vercel (Frontend) + Railway (Backend)	Production	Global edge delivery with automated CI/CD from GitHub.

3. Architecture & Data Flow



4. Pipeline Stages: Step-by-Step

Stage 1: File Ingestion & Structural Inspection

- The user selects or drags Excel (`.xlsx`, `.xls`) or CSV files into the **Data Ingestion** studio.
- `POST /api/upload/inspect` examines the workbook headers, row counts, and sheet roles without writing to the database.
- Files named with `"consolidated"` are tagged as `Pre-Consolidated File` to avoid re-processing redundant master registers.

Stage 2: Canonical Column Mapping

- The engine maps raw source headers (e.g., `Client Name`, `Mobile Number`, `Flat No`, `Gross Area`) against **23 standard fields**.
- Users can review or override mappings in the interactive **Header Remapping Studio** before starting the job.

Stage 3: Data Cleaning & Normalization

- Phone Numbers:** Normalizes international and UAE numbers to standard E.164 formats (`+971...`, `+1...`, `+44...`).
- Surface Area:** Automatically converts square meters to square feet using the exact ratio $1 \text{ m}^2 = 10.7639 \text{ sq.ft}$.
- Text & Casing:** Strips noise characters, unifies letter casing, and decouples buyer names from corporate builder names.

Stage 4: Strict Outreach-Readiness Validation

- VALID (Outreach Ready):** Record possesses **both** a verified person/entity name **and** at least one usable contact detail (Mobile 1, Mobile 2, Mobile 3, or Email).
- INCOMPLETE:** Record has a name but no contact details, or contact details without a name. Filtered out of default views.
- INVALID:** Record failed structural parsing or lacked basic identifiers.

Stage 5: Master Developer & Community Enrichment

- When a record lacks a developer (common in 89% of individual property registers), the engine matches `Community`, `Sub-Community`, or `Project` against the **UAE Development Builders Reference** (483 communities across all 7 emirates).
- Example: `Dubai Hills - Park` → Auto-populates `Developer = Emaar Properties (JV with Meraas/Dubai Holding)`.
- Features an embedded JSON fallback (`engine/resources/uae_developers.json`) ensuring 100% availability in cloud deployments.

Stage 6: Deterministic Deduplication

- Calculates a SHA-256 `identity_hash` based on normalized location (`community + building + unit`) and identity (`name + mobile_1`).
- Eliminates duplicate entries across sheets and across batches without relying on unstable row indices.

Stage 7: Memory-Bounded Batch Persistence

- Ingests in chunks of 500–1,000 rows within isolated database transactions.
- If a chunk fails, only that batch rolls back while previous successful batches remain intact.

5. User Interface Features

1. **Neumorphic Auth Lock Screen:**
 2. Gated with access key `dev123`.
 3. Shake animation on incorrect entry; automatic session persistence in `localStorage`.
 4. **Light & Dark Theme Converter:**
 5. Dynamic toggle with persistent high-contrast typography (`#F8FAFC` headings, `#F1F5F9` body) ensuring 100% visibility in dark mode.
 6. **Interactive Search & Multi-Column Filters:**
 7. Live free-text search across Name, Community, Unit, Mobile, Developer, and Plot.
 8. Categorical dropdowns for Community, Property Type (Residential, Commercial, Land), Bedroom count, and Status.
 9. **Excel & CSV Export Engine:**
 10. Dedicated `Export Excel (.xlsx)` and `Export CSV (.csv)` buttons.
 11. Preserves active search queries and filter parameters — exports only the records matching the user's view.
-

6. Verification & Health Audit

The entire codebase passes all verification criteria: - **FastAPI / Uvicorn Server:** Running and healthy at `http://127.0.0.1:8001/docs`. - **Vite / React Client:** Production build succeeds in < 1 second. - **Database Connectivity:** Verified connection to Supabase PostgreSQL pooler. - **Data Integrity Test:** Automated health check script (`scripts/health_check.py`) confirms 100% pass rate across enrichment, validation, and schema definitions.
